

HOWTO

CONTINUOUS PERFORMANCE ENGINEERING

Henrik Ingo

Fosdem 2026

HOWTO

CONTINUOUS BENCHMARKING

Henrik Ingo

Fosdem 2026

POLL

Continuous Performance
Engineering
maturity levels

1. We don't really do benchmarking
2. Benchmarks yes, continuous no
3. Some benchmarks run in CI
...but we ignore the results
4. Benchmarks in CI,
Daily or more often,
Actionable results,
Regressions fixed within weeks

CONTINUOUS INTEGRATION

CONTINUOUS TESTING

CONTINUOUS DEPLOYMENT

DEVOPS

LEFT SHIFTING

ITERATIONS

CONTINUOUS PERFORMANCE ENGINEERING?

Somewhere in that evolution, they forgot to bring the performance engineer



Velociraptor, the fastest dinosaur (wikipedia)

WHY WAS PERFORMANCE ENGINEERING LEFT IN 1999?

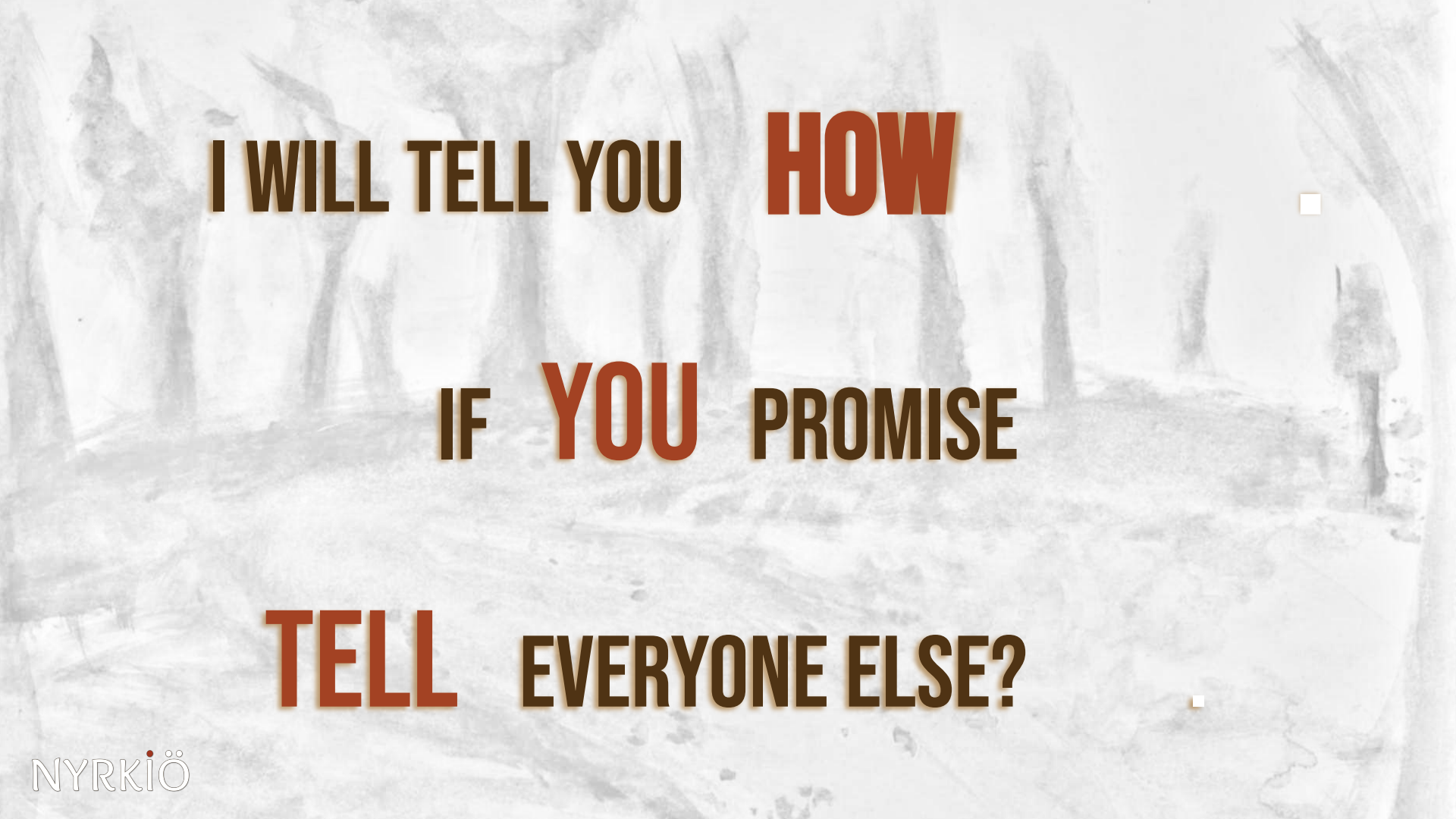
Deploying is necessary - benchmarking is optional

Performance Engineers busy fixing prod / customer problems

....and maybe enjoying it?

Math is hard

Relevant tuning is unintuitive, not well known



I WILL TELL YOU HOW

IF YOU PROMISE

TELL EVERYONE ELSE?

AGENDA

1. Benchmarking
Make it Continuous
2. Change Point Detection
Math & Science
3. Minimizing noise in the
Benchmarks
Assume nothing.
Measure everything.

BENCHMARKING PROCESS AND TOOLS

ON BENCHMARK TOOLS & DESIGN

Previous talk by Kemal Akkoyun & Augusto de Oliveira.

But in general...

- at this point each language has its own standard framework:
 - Java & JMH
 - Python & pytest-benchmark
 - etc...

Frameworks to run fully, end to end automated **distributed** benchmarks exist. Personally I believe we will see new innovation coming here. (k8s)

GITHUB-ACTION-BENCHMARK

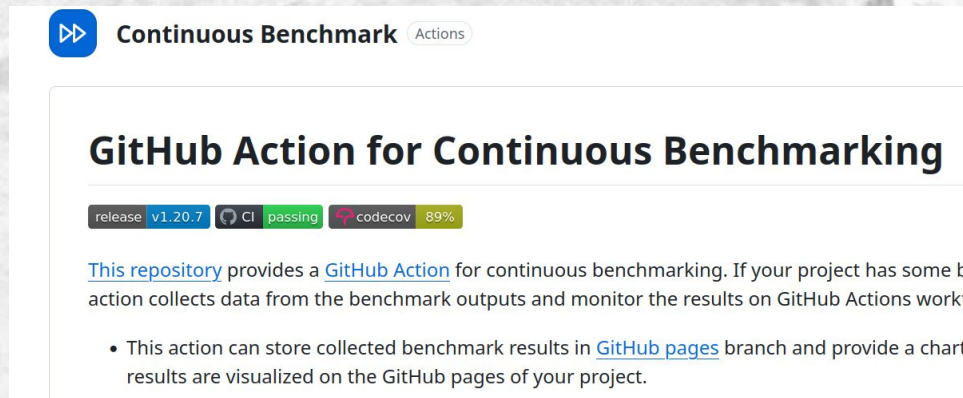
Use in your workflows immediately after the benchmark step

Supports output from all major frameworks

Stores result history in your github repo.

Threshold based alerts.

Default threshold = 100% (2x)



The screenshot shows the GitHub repository page for 'Continuous Benchmark' by 'Actions'. The repository name is 'Continuous Benchmark' and it is categorized under 'Actions'. The repository has a 'release v1.20.7' badge, a 'CI' badge, a 'passing' badge, a 'codecov 89%' badge, and a 'GitHub Actions' badge. The repository description states: 'This repository provides a GitHub Action for continuous benchmarking. If your project has some benchmarking action, this action collects data from the benchmark outputs and monitor the results on GitHub Actions workflow pages.' A list of features is provided: 'This action can store collected benchmark results in [GitHub pages](#) branch and provide a chart of the results are visualized on the GitHub pages of your project.'

Continuous Benchmark Actions

GitHub Action for Continuous Benchmarking

release v1.20.7 CI passing codecov 89%

This repository provides a [GitHub Action](#) for continuous benchmarking. If your project has some benchmarking action, this action collects data from the benchmark outputs and monitor the results on GitHub Actions workflow pages.

- This action can store collected benchmark results in [GitHub pages](#) branch and provide a chart of the results are visualized on the GitHub pages of your project.



CHANGE POINT DETECTION

A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.

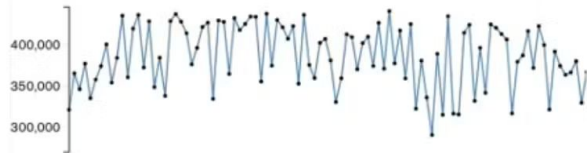
▼ insert_ttl-wiredTiger

c48f298

ops per sec:

313,407

Compare



40 % noise.
No regression!

▶ insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298

ops per sec:

492,207

Compare



17 %
No regression!

▶ insert_vector_secondary_load_phase-wiredTiger

▶ insert_vector_secondary_overall-wiredTiger

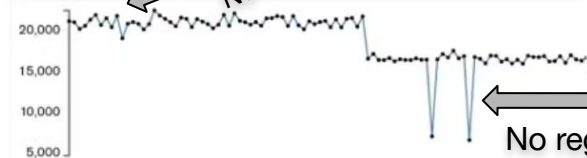
▶ word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298

ops per sec: 5,575

Compare



15 %
No regression!

70%
No regression!

A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.

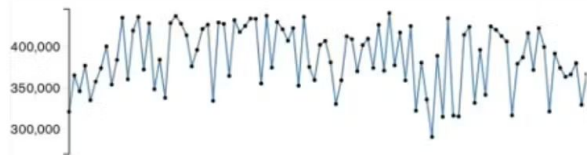
▼ insert_ttl-wiredTiger

c48f298

ops per sec:

313,407

Compare



40 % noise.
No regression!

▶ insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298

ops per sec:

492,207

Compare



17 %
No regression!

▶ insert_vector_secondary_load_phase-wiredTiger

▶ insert_vector_secondary_overall-wiredTiger

▶ word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298

ops per sec: 5,575

Compare



Regression!

70%
No regression!

EVERYONE ON GITHUB, 2025:

SELECT 1

time ↓

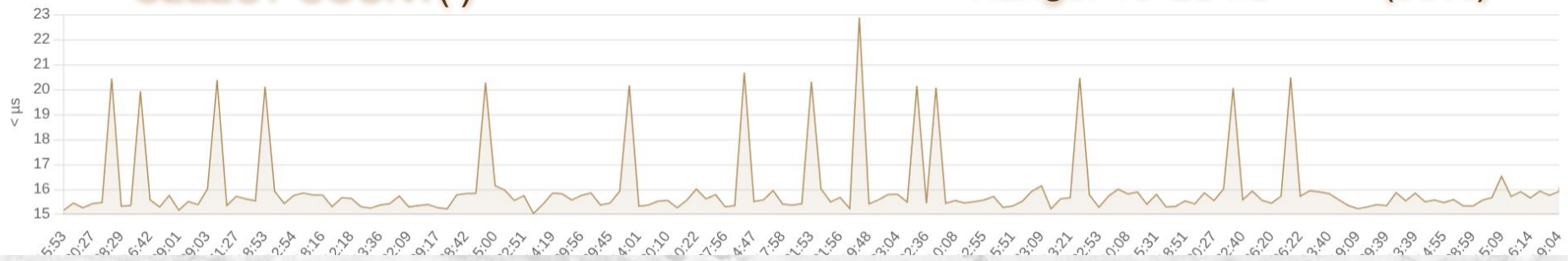
Range: 60-100 ns (40%)



SELECT COUNT(*)

time ↓

Range: 15-23 ns (50%)

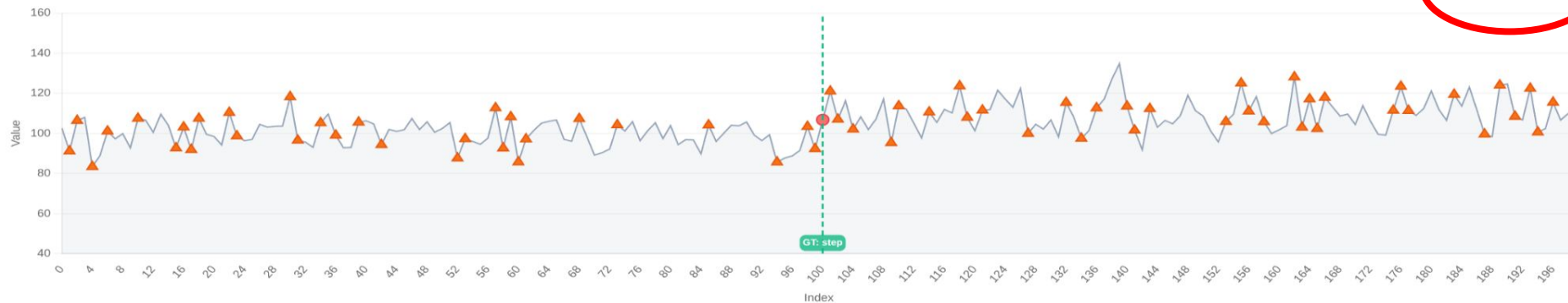


AUTOMATION IS ONE OF THE CORE PRINCIPLES...

Threshold based alerting...

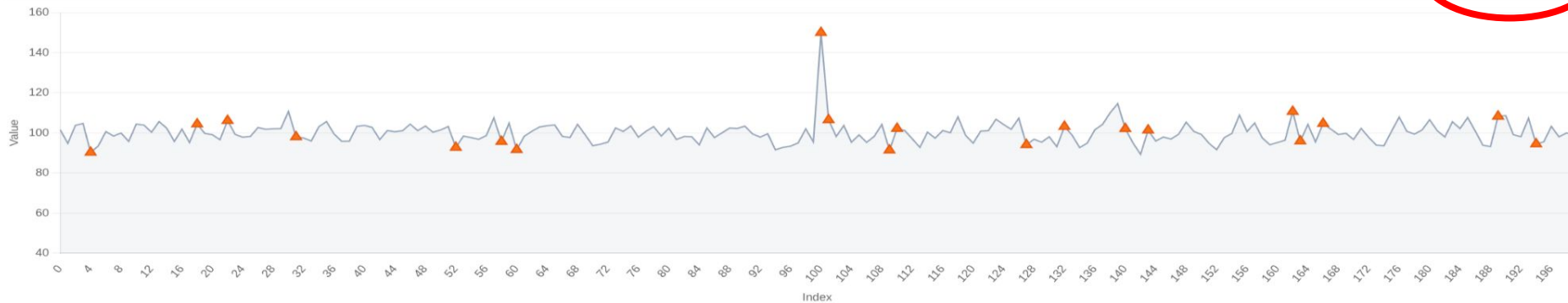
■ Threshold Alert (>9.6%, offset=1)

1 TP / 65 FP



■ Threshold Alert (>9.6%, offset=1)

0 TP / 20 FP



E-DIVISIVE MEANS (MATTESON & JAMES, 2014)

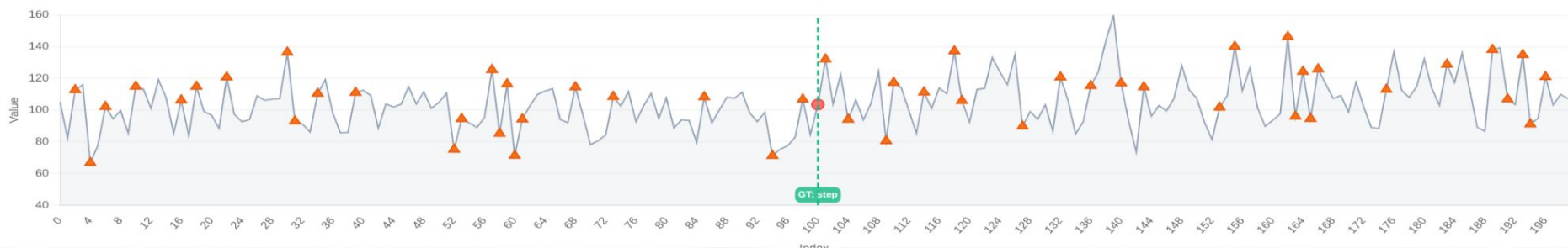
■ Otava Analysis

1 TP / 0 CM / 0 FP



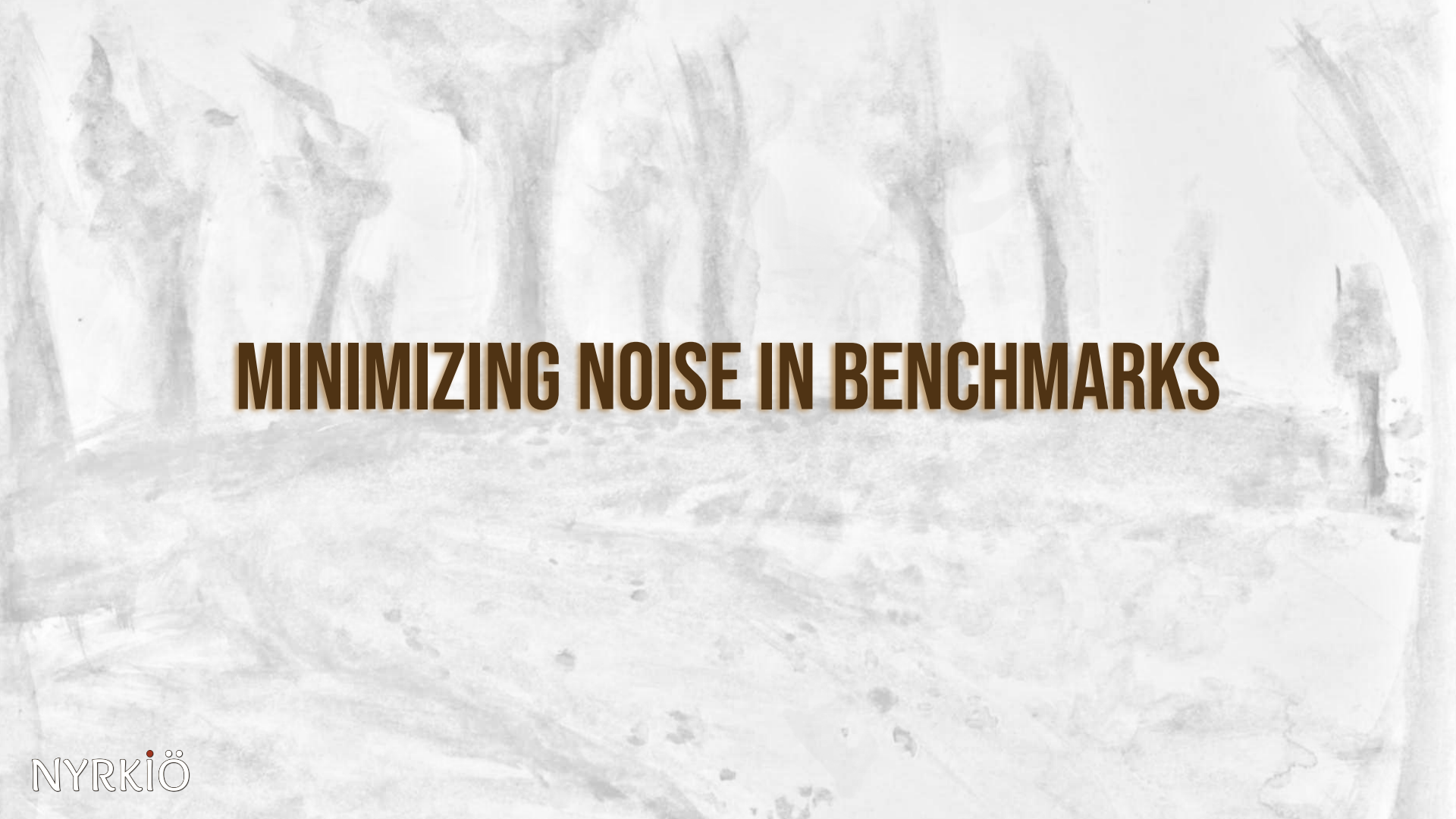
■ Threshold Alert (>22.1%, offset=1)

1 TP / 0 CM / 49 FP



A soft, watercolor-style illustration of a forest scene. The background is filled with various shades of grey, brown, and white, creating a misty or ethereal atmosphere. Several tree trunks are visible, some with sparse foliage. The overall texture is painterly and delicate.

otava.apache.org



MINIMIZING NOISE IN BENCHMARKS

REASONS WHY BENCHMARK RESULTS ARE SO NOISY?

CLOUD

CLOUD

?

NOISY NEIGHBOR

SOVEREIGN
CLOUD

BAD PROGRAMMERS

FUN EXERCISE: RETROACTIVELY LIST ASSUMPTIONS BUILT INTO YOUR CURRENT ARCHITECTURE

Dedicated instance = more stable performance

Placement groups minimize network latency & variance

Different availability zones have different hardware

For write heavy tests, noise comes from disk

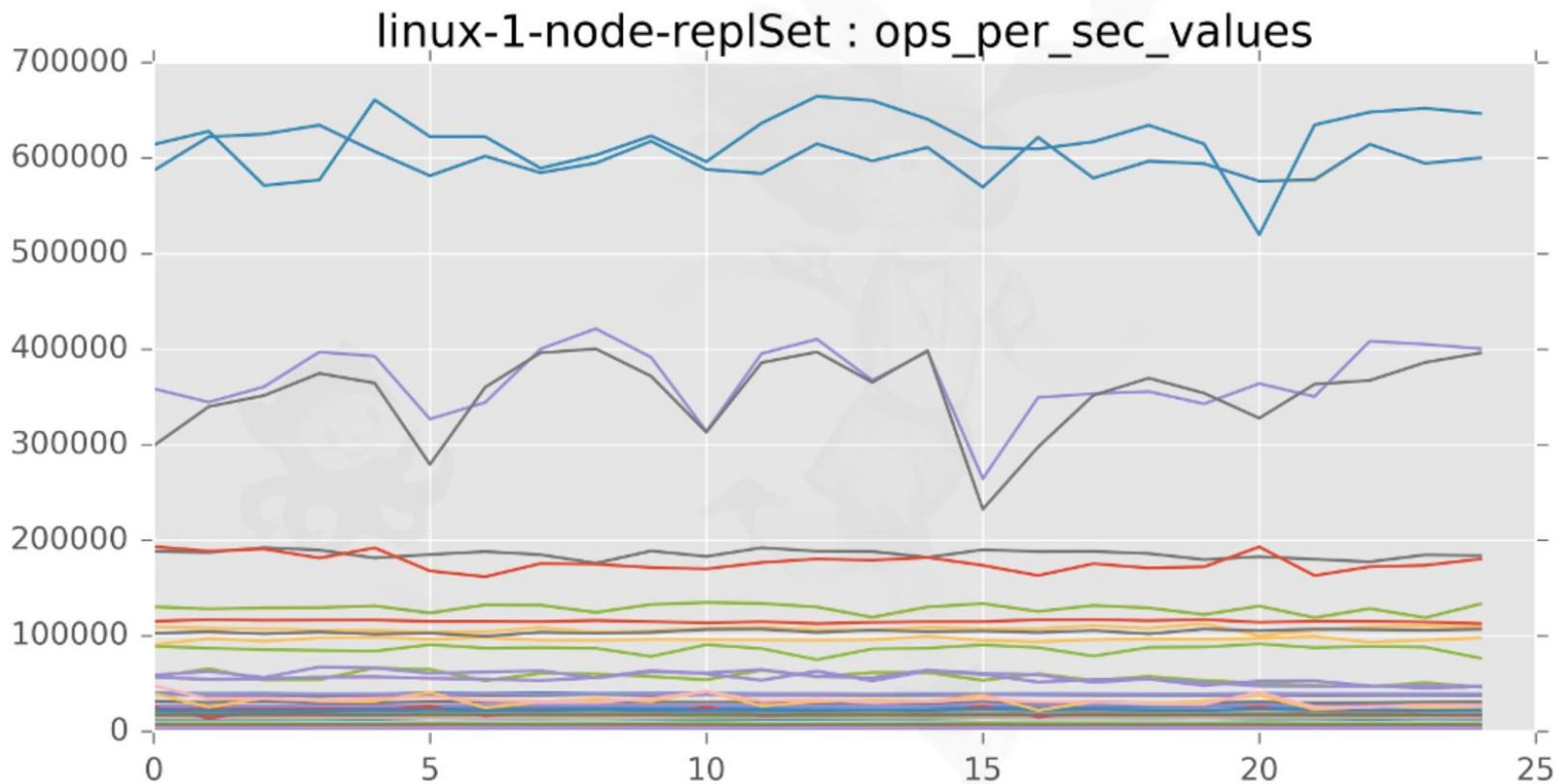
Ephemeral (SSD) disks have least variance

There are good and bad EC2 instances

Just use i2 instances (better SSD)

You can't use cloud for performance testing

1 MONGODB BINARY, 5 SERVERS, REPEAT TESTS 5X (2017)



FUN EXERCISE: RETROACTIVELY LIST ASSUMPTIONS BUILT INTO YOUR CURRENT ARCHITECTURE

Dedicated instance = more stable performance

Placement groups minimize network latency & variance

Different availability zones have different hardware

For write heavy tests, noise comes from disk

Ephemeral (SSD) disks have least variance

There are good and bad EC2 instances

False

Just use i2 instances (better SSD)

You can't use cloud for performance testing

WAS IT THE NEIGHBORS?

Continuous Benchmarking is hard because...

- Your hardware is actively working against you:
 - CPU Frequency scaling
 - CPU boost
 - HyperThreading...
 - NUMA architecture...

```
man cpupower
```

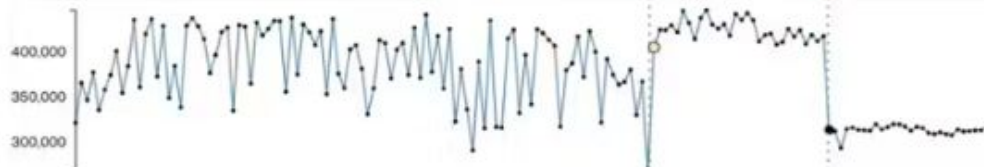
...and that was just the CPU!

Pro Tip: Have you noticed how on EC2 the *local SSD* disks are not actually called that in AWS documentation.

▼ insert_tti-wiredTiger

c48f298
ops per sec:
313,407

Compare



► insert_capped_indexes-wiredTiger

▼ insert_vector_primary-wiredTiger

c48f298
ops per sec:
492,207

Compare



► insert_vector_secondary_load_phase-wiredTiger

► insert_vector_secondary_overall-wiredTiger

► word_count_1M_doc-wiredTiger

▼ insert_jtrue-wiredTiger

c48f298
ops per sec: 5,575

Compare



SSD -> EBS



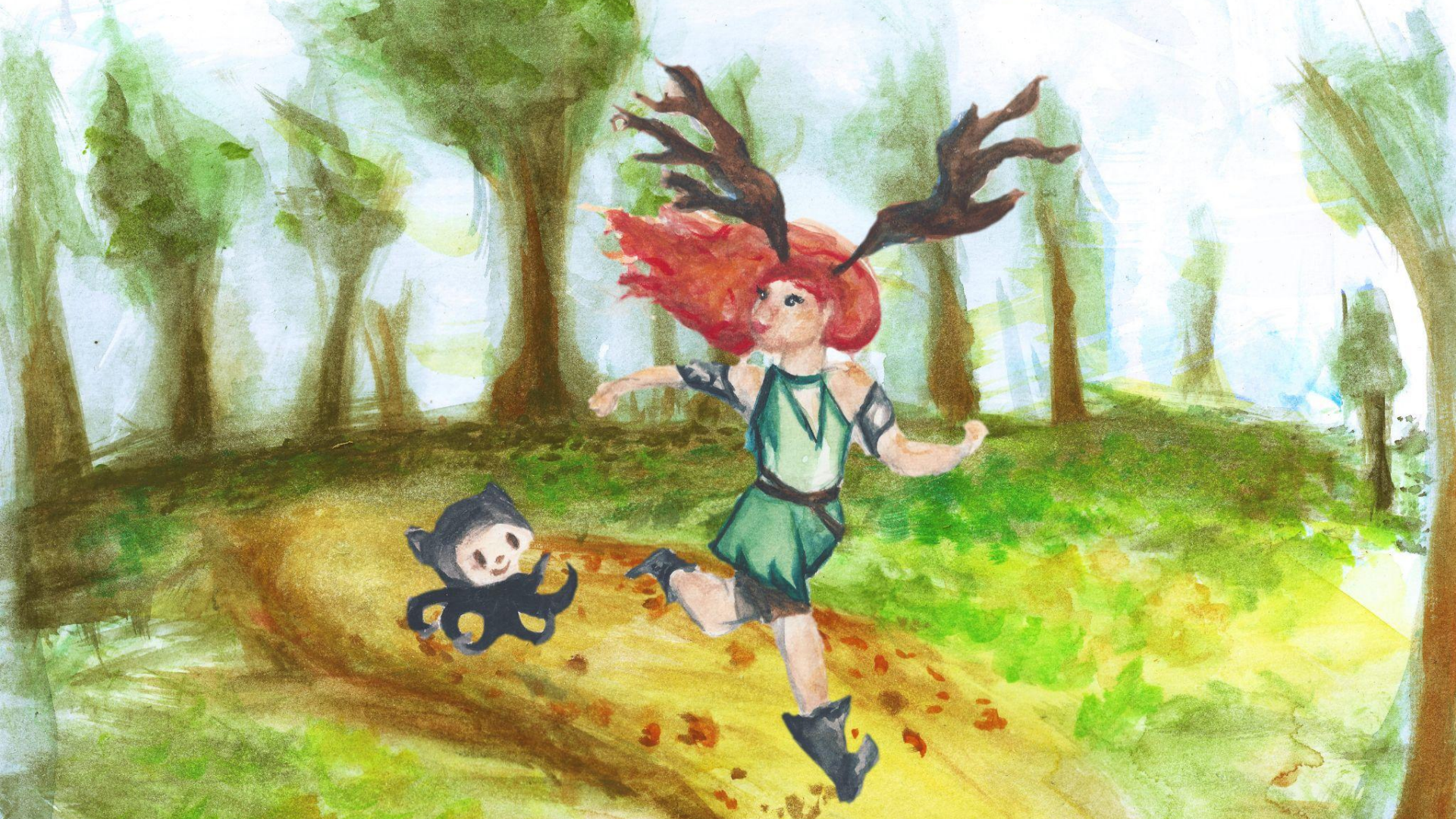
CPU: No HT, single
socket, scheduling

Noise range for all tests:
5%



10 years later...

NYRKIÖ



NYRKIÖ

GitHub

Runners



NYRKIÖ

GitHub

Runners



1. Install as GitHub app:
nyrkio.com
2. Pick a subscription (20 c/h)
3. In workflow.yml:

```
runs-on : nyrkio_2
```

C7a instances
Hand picked &
Carefully tuned

NOT best performance
NOT for price/performance

But

REPEATABLE
performance

NYRKIÖ

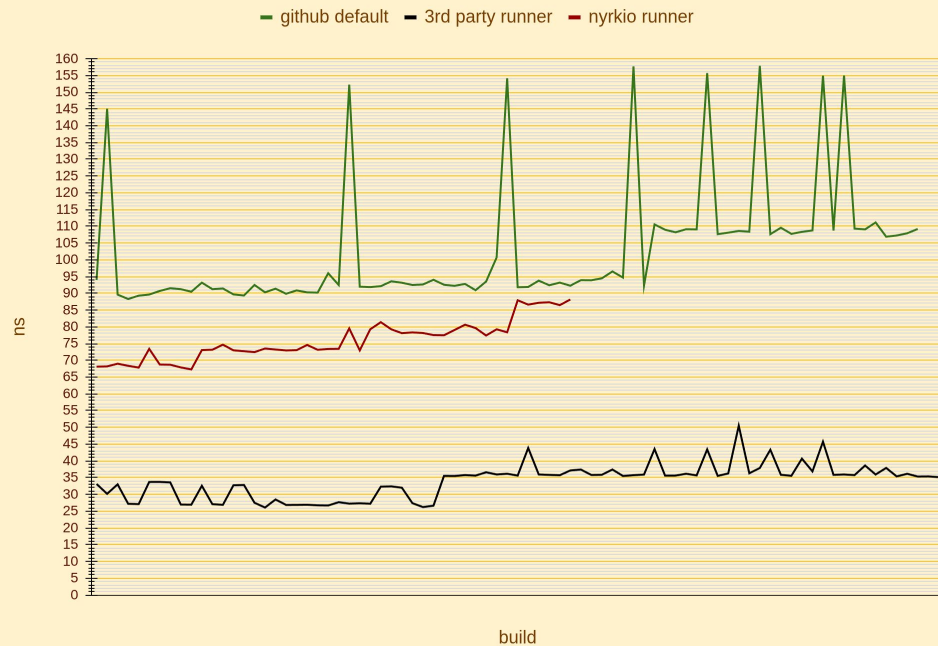
GitHub

Runners

min-max ranges (ns)

github default	3rd party runner	nyrkio runner
88	35	73
154	51	75
66 ns	15 ns	2 ns
75%	44%	3%

SELECT 1



NYRKIÖ

GitHub

Runners

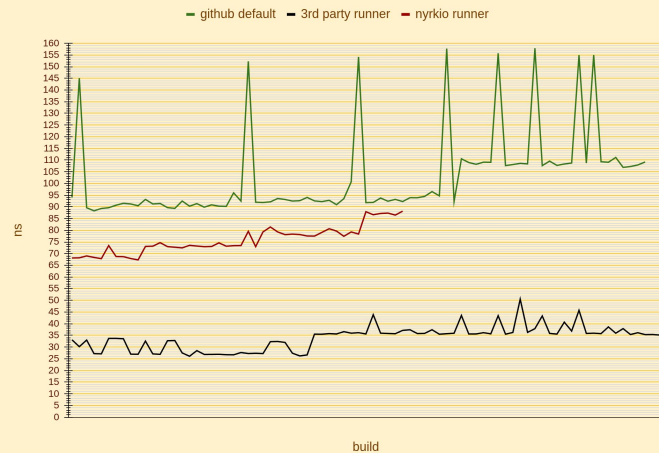
min-max ranges (ns)

github default	3rd party runner	nyrkio runner
88	35	73
154	51	75
66 ns	15 ns	2 ns
75%	44%	3%

min-max ranges (ns)

github default	3rd party runner	nyrkio runner
18	49	11
25	59	13
6 ns	10 ns	1 ns
30%	21%	12%

SELECT 1



SELECT COUNT(*)



ANY DIFFERENCE IN 10 YEARS?

C7a family offers high fidelity performance for 100% of the price

- No hyperthreading
- Single CPU socket
- AMD?

> 4x cheaper

Future opportunities:

- sub nano-second precision
- Dist-sys: Run K8s cluster, on 128 core server

WHAT HAVE WE LEARNED?

1.

Start simple
Benchmark ***continuously***

3. Performance tuning for
repeatable results is
counter intuitive.

2.

Apply

Math & Science

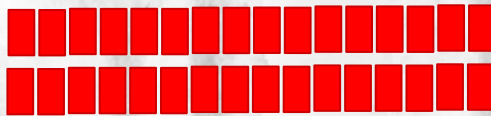
... until .
false positives
no longer hurt



PS: CANARIES

Add tests that measure your infrastructure.

- CPU
- Disk
- Network



▼ canary_server-cpuloop-10x

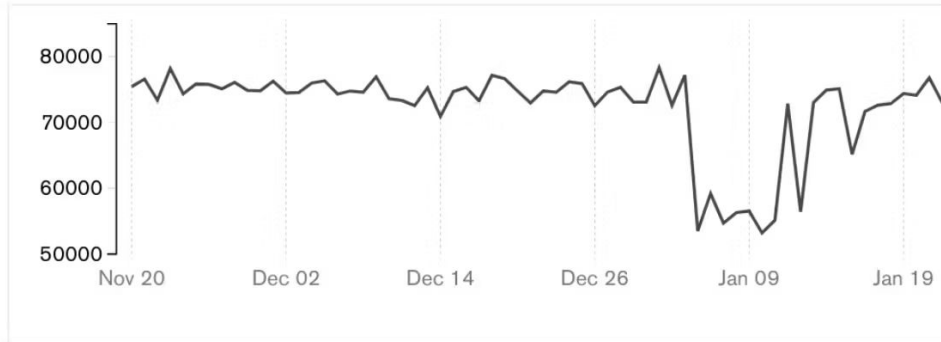
78b96cb

Jan 30 2018

ops per sec:

71,552

[Compare](#)



CREDITS AND REFERENCES

Nyyrikki and cat running, watercolor: Ebba Ingo

Velociraptor: Wikimedia commons

Otava test data graphs:
Joe Drumgoole & Claude

Everyone who contributed to Change Detection,
now known as Apache Otava (incubating)

Ingo,Daly: Reducing Variability in Performance
Tests on EC2

www.youtube.com/watch?v=3kHGZ7niHI4

David Daly et.al.: The Use of Change Point
Detection to Identify Software Performance
Regressions in a Continuous Integration
System, 2020.

Fleming & Kołaczowski: Hunter: Using Change
Point Detection to Hunt for Performance
Regressions, 2023.

Fixing Performance Regressions Before they
Happen (Netflix)

8 Years of Optimizing Apache Otava: How
disconnected open source developers took an
algorithm from n^3 to constant time

Processor state control for Amazon EC2 Linux
instances